

ARMOR: A Robust Self-Supervised Framework for Root Cause Analysis in Microservices under Missing Modality

Anonymous Author(s)

Abstract

Automated incident management is critical for microservice reliability. While recent unified frameworks leverage multimodal data for joint optimization, they unrealistically assume perfect data completeness. In practice, network fluctuations and agent failures frequently cause missing modalities. Existing approaches relying on static placeholders introduce imputation noise that masks anomalies and degrades performance. To address this, we propose ARMOR, a robust self-supervised framework designed for missing modality scenarios. ARMOR features: (i) a modality-specific asymmetric encoder that isolates distribution disparities among metrics, logs, and traces; and (ii) a missing-aware gated fusion mechanism utilizing learnable placeholders and dynamic bias compensation to prevent cross-modal interference from incomplete inputs. By employing self-supervised auto-regression with mask-guided reconstruction, ARMOR jointly optimizes anomaly detection (AD), failure triage (FT), and root cause localization (RCL). AD and RCL require no fault labels, while FT relies solely on failure-type annotations for the downstream classifier. Extensive experiments demonstrate that ARMOR achieves state-of-the-art performance under complete data conditions and maintains robust diagnostic accuracy even with severe modality loss.

CCS Concepts

• Software and its engineering → Software maintenance tools.

Keywords

Microservice Systems, Anomaly Detection, Failure Triage, Root Cause Localization, Missing Modality

ACM Reference Format:

Anonymous Author(s). 2018. ARMOR: A Robust Self-Supervised Framework for Root Cause Analysis in Microservices under Missing Modality. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Microservice architectures have become the foundational infrastructure for modern cloud-native applications due to their inherent scalability and agility [13, 36]. However, the massive scale and dynamic complexity of these distributed systems make performance

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

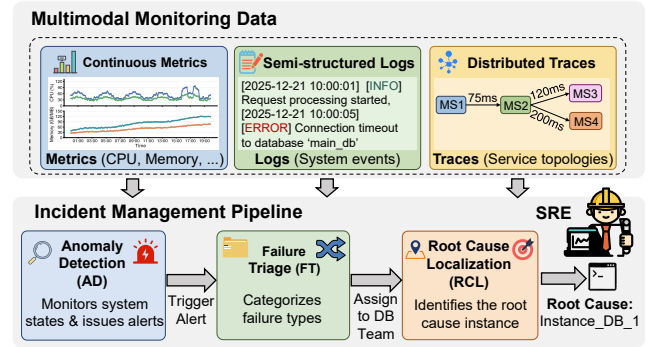


Figure 1: Overview of the multimodal incident management pipeline. Diverse observability data (metrics, logs, and traces) drives three sequential tasks: AD, FT, and RCL.

anomalies and system failures inevitable, frequently leading to severe service disruptions and substantial financial losses [30]. Consequently, ensuring system reliability through automated incident management has become a priority for site reliability engineers (SREs) [43].

To manage incidents effectively, SREs rely heavily on multimodal monitoring data, including continuous metrics, semi-structured logs, and distributed traces, which collectively capture the overall system state [17, 29, 40, 42]. As Figure 1 illustrates, these diverse data streams sequentially drive a standard incident management pipeline comprising three diagnostic tasks: anomaly detection (AD) [4–6, 8, 11, 15, 16, 18, 52], failure triage (FT) [23, 24, 37, 38, 44], and root cause localization (RCL) [14, 22, 32, 35, 39]. Initially, AD continuously monitors system states to trigger alerts upon detecting abnormal behaviors. Subsequently, FT categorizes the detected anomalies into specific failure types for appropriate engineering teams. Finally, RCL identifies the exact culprit instance responsible for the failure.

Current intelligent incident management approaches generally fall into two categories. Early methods rely on isolated, single-task techniques for different diagnostic stages [18, 20, 44]. However, decoupling these related tasks ignores the shared diagnostic context embedded in system deviations, often leading to redundant maintenance overhead, inefficient resource utilization, and delayed mitigation responses [29]. To address these limitations, recent research shifts toward unified multi-task frameworks [17, 29, 40, 42]. Instead of maintaining separate pipelines, these methods extract shared knowledge from multimodal monitoring data to jointly optimize AD, FT, and RCL. By modeling inherent dependencies across these stages, such end-to-end solutions achieve significant performance improvements and streamline troubleshooting under ideal conditions.

Despite their success, existing unified frameworks [17, 29, 40, 42] assume that the collected multimodal monitoring data is perfectly aligned and complete. However, in real-world production environments, observability infrastructure vulnerabilities, such as network fluctuations, telemetry agent crashes, and configuration errors, frequently cause missing modalities regardless of the actual microservice status [10, 53]. When processing incomplete data, current models typically fuse the available inputs with static placeholders [3, 50], causing significant performance degradation. Although supervised approaches attempt to handle incomplete modalities, the extreme scarcity of industrial fault labels renders them prone to overfitting and difficult to deploy. Adopting a self-supervised approach provides a practical alternative to eliminate reliance on manual annotations. Nevertheless, developing a robust, self-supervised incident management framework capable of handling missing modalities requires addressing three primary challenges:

(1) **Tightly coupled encoding makes any modality absence corrupt surviving signals.** When observability infrastructure fails, multiple modalities may become unavailable at the same time, and SREs cannot predict which combination will drop next. Existing methods tightly couple metrics, logs, and traces during early representation learning, so missing modalities corrupt the representations built from the survivors [17, 40, 42, 52]. The structural asymmetry between dense continuous metrics and sparse discrete logs and traces makes this worse: encoding heterogeneous signals with a shared architecture forces the surviving modalities into a representation space shaped by the absent ones, losing the diagnostic clues they actually carry.

(2) **Static imputation masks real failures by mimicking a healthy idle state.** When a Prometheus agent goes offline, filling its missing channels with zero tells the model that every monitored resource sits at exactly zero consumption [41, 50]. That is precisely the signature of a healthy idle service, not a failing one. Conventional fusion mechanisms [29, 42, 52] adopt rigid early concatenation and rely on static imputation [3, 50] for structural consistency. The resulting pseudo-normal signal overwhelms the anomaly indicators captured by surviving logs and traces, suppressing the deviations that SREs would otherwise use to diagnose the incident.

(3) **Distorted representations break unified diagnosis across all three tasks.** Without a representation that holds up under missing data, SRE teams lose the ability to run a single unified AD, FT, and RCL pipeline. Each missing modality scenario produces a different distortion in the fused representation, forcing teams back to maintaining separate per-modality diagnostic configurations and reintroducing the fragmentation overhead that unified frameworks were designed to eliminate. Self-supervised models derive diagnostic signals from predictive errors [39, 42, 52], but static imputation biases those errors toward normal states. Deriving a stable failure signature under these distorted inputs, one that simultaneously supports all three tasks without manual annotations, remains an unresolved problem.

To address these limitations, we propose ARMOR, an Automated and Robust framework handling Missing modality for Root cause analysis in microservices. Specifically, ARMOR comprises three core modules designed to systematically resolve the aforementioned challenges: (1) *Modality-specific status learning*. We design an asymmetric encoder tailored to the distinct distribution properties of

continuous metrics and sparse events. This decoupled architecture hierarchically disentangles temporal, channel, and spatial dependencies, preventing the absence of a single modality from corrupting the representations of the others. (2) *Missing-aware global fusion*. We introduce an attention-guided gating mechanism incorporating learnable placeholders and targeted biases. Instead of employing static imputation, this module explicitly recognizes absent inputs and adaptively compensates for their routing contributions, preventing static placeholders from inducing cross-modal interference. (3) *Online diagnosis via unified representations*. ARMOR constructs a robust unified representation by concatenating explicit reconstruction errors with latent system embeddings. This comprehensive signature establishes a shared foundation to concurrently support AD, FT, and RCL, even under severe modality dropouts and strict label scarcity. The main contributions of this work are as follows.

- We present ARMOR, which, to the best of our knowledge, is the first self-supervised incident management framework explicitly designed to handle missing modalities in microservices, enabling label-free AD and RCL, with FT requiring only failure-type annotations for the frozen-encoder classifier.
- We devise a modality-specific asymmetric encoder and a missing-aware gated fusion mechanism. By leveraging learnable placeholders and dynamic bias compensation, ARMOR effectively mitigates imputation noise and prevents spurious correlations caused by incomplete inputs.
- Extensive experiments on benchmark microservice systems demonstrate that ARMOR achieves state-of-the-art performance under complete data settings and maintains superior diagnostic stability even under severe modality loss, significantly outperforming existing baselines.

2 Motivation

This section motivates the proposed framework by exploring the gap between ideal academic assumptions and complex industrial realities. Specifically, we examine the inevitability of missing modalities in production environments and demonstrate how current multimodal fusion strategies remain fundamentally vulnerable to such incomplete data.

2.1 Why Are Missing Modalities Inevitable in Production Environments?

Existing research on unified modeling relies heavily on the assumption of perfectly complete and aligned multimodal datasets (e.g., metrics, logs, and distributed traces) [17, 29, 42]. However, such ideal conditions rarely exist in industrial cloud-native environments. Due to the scale and dynamic complexity of microservice architectures, the absence of modalities is a common occurrence rather than an infrequent edge case. This data absence is primarily driven by vulnerabilities within the observability infrastructure [3, 41, 50, 53] (e.g., data collection and transmission pipelines) rather than by failures within the business microservices themselves.

Figure 2 illustrates how practical factors, such as agent failures and network policies, cause abrupt data gaps even when the underlying services remain healthy. For example, telemetry agents (e.g., Prometheus node_exporter) operate in resource-constrained

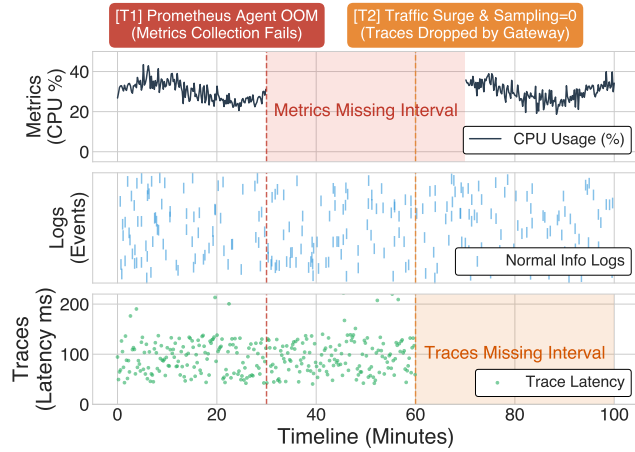


Figure 2: The generation of missing modalities in production environments. This timeline illustrates how vulnerabilities in the observability infrastructure (e.g., agent crashes or aggressive sampling) cause abrupt data loss, even while the underlying microservice operates normally and outputs logs.

environments and are highly susceptible to out-of-memory errors due to high monitoring overhead. At timestamp T1, an agent crash causes a sudden and prolonged absence of metrics; however, the continuous output of normal information logs confirms that the business service continues to process requests without interruption. Similarly, network fluctuations and aggressive sampling strategies create substantial data gaps. During extreme traffic surges, operations teams routinely downgrade the observability pipeline to preserve bandwidth for core transactions. At timestamp T2, this configuration results in traces being actively dropped by the gateway, yet the persistent stream of logs again verifies the operational stability of the service. Beyond such runtime interruptions, configuration errors and system heterogeneity hinder data generation. Modern systems frequently incorporate legacy services or third-party components that lack proper instrumentation, leading to an inherent absence of specific modalities without triggering explicit business errors [3, 41, 50, 53].

As Figure 2 shows, missing modalities occur as complete channel-level outages that arrive irregularly across different services and time steps, primarily due to collection and transmission issues. Because extreme traffic and resource contention often precede actual system failures, the absence of observational data frequently coincides with the exact incidents that site reliability engineers must diagnose.

Finding 1: In industrial microservices, observability infrastructure vulnerabilities produce complete modality-level blackouts: a process crash or network failure can eliminate an entire data stream instantaneously. Since such outages frequently coincide with actual system failures, methods requiring perfectly complete data are highly impractical.

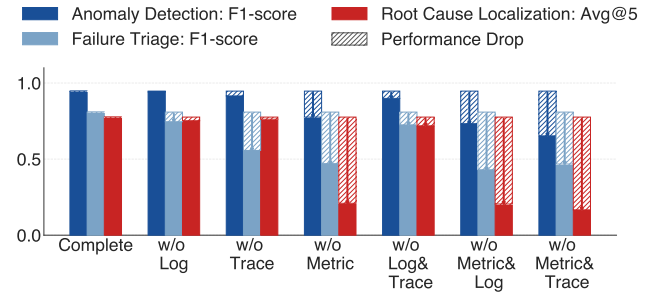


Figure 3: Performance degradation of ART [42], the strongest self-supervised unified baseline, under missing modalities. Metric loss causes a steep drop in RCL accuracy, confirming that static imputation cannot compensate for absent continuous streams.

2.2 How Do Existing Frameworks Perform with Missing Modalities?

Given the inevitability of missing modalities in distributed systems, evaluating how state-of-the-art unified frameworks [17, 40, 42] handle incomplete inputs is critical for practical deployment. We probe ART [42], the best-performing self-supervised unified baseline under complete data, as the representative: as Figure 3 illustrates, it experiences severe and asymmetric performance degradation under missing modalities. Although the framework maintains moderate resilience when discrete logs are missing, the absence of continuous metrics causes a significant decline, particularly in the accuracy of RCL. This result confirms that conventional static imputation fails to bridge the semantic gap left by missing modalities, leaving the automated monitoring pipeline highly vulnerable. The degradation is most severe when metrics are absent: unlike logs and traces, which record discrete events at coarse granularity, metrics provide dense, continuous resource measurements across every time step, so their loss leaves the surviving modalities with far less signal to compensate.

This systemic vulnerability stems from the rigid nature of early-concatenation fusion mechanisms. When a specific modality becomes unavailable, existing methods routinely employ static imputation (e.g., filling missing dimensions with default constants) to maintain structural consistency [3, 10, 53]. Although computationally convenient, this naive alignment entirely ignores semantic validity. Consequently, high-dimensional neural networks mistakenly interpret these deterministic placeholders as high-confidence features indicating a stable operational state.

This pervasive “pseudo-normal” signal introduces severe cross-modal interference. As the t-SNE projection in Figure 4 demonstrates, static imputation forcefully shifts the latent features of anomalous instances, which otherwise form distinct and separable clusters, directly into the dense cluster of normal states. This mathematical distortion heavily suppresses the subtle anomalous deviations successfully captured by the remaining available modalities, thereby systematically reducing overall diagnostic accuracy. Therefore, eliminating this imputation-induced distortion is a prerequisite for robust incident management.

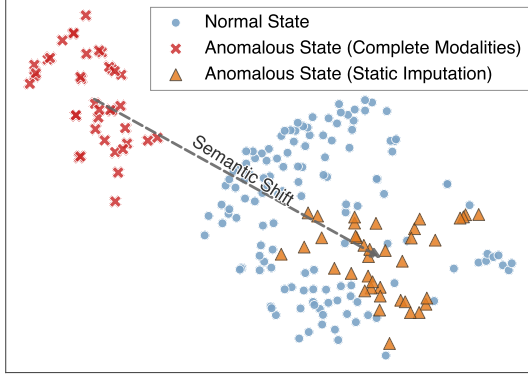


Figure 4: The impact of static imputation on latent feature distributions. A t-SNE visualization demonstrates how non-adaptive default values force the features of anomalous instances into the normal cluster, creating spurious correlations that suppress actual diagnostic signals.

Finding 2: The naive imputation of missing modalities biases joint representations toward normal states, severely degrading diagnostic performance and necessitating the development of robust, missing-aware fusion mechanisms.

3 Preliminaries

3.1 Self-supervised Learning

Self-supervised learning derives supervisory signals directly from unlabeled data via pretext tasks, such as reconstruction [31, 33], to learn generalizable representations. In microservice incident management, acquiring manual fault annotations across large-scale environments is prohibitively expensive and labor-intensive [42]. Self-supervised learning mitigates this challenge by eliminating reliance on extensive labeled data. Unlike traditional supervised joint learning requiring complex loss balancing [10, 17], self-supervised learning naturally facilitates unified multi-task modeling. By learning generalized failure semantics, a single self-supervised pipeline seamlessly supports AD, FT, and RCL at the representation level [42]. Therefore, we adopt it as our foundational methodology.

3.2 Missing Modalities in Multimodal Fusion

While self-supervised unified frameworks have advanced microservice incident management, they universally assume complete modality availability [29, 40, 42], leaving them ill-equipped for missing modality scenarios common in production deployments. The multimodal learning community offers two categories of solutions [47]. Generative imputation [19, 21] synthesizes missing channels but incurs substantial computational overhead and risks fabricating false diagnostic signals. Representation-level methods [28, 48] adjust modality contributions via dynamic routing yet often depend on a primary modality for alignment, failing when key signals drop.

Applying these solutions to microservice observability is complicated by a structural asymmetry absent from standard multimodal tasks. Unlike domains with symmetric latent spaces [47], microservice data comprises dense, continuous metrics and sparse, discrete logs and traces, whose heterogeneous nature makes uniform imputation strategies unreliable. When metrics are missing, static placeholders such as zero-filling are misread by neural networks as pseudo-normal signals, causing distribution shifts that suppress anomaly indicators in the surviving modalities [48]. A reliable framework must therefore isolate modality-specific semantics prior to fusion to prevent imputation noise from corrupting the diagnostic representation.

3.3 Problem Formulation

Let $\mathcal{I} = \{1, \dots, N\}$ denote the set of N interconnected microservice instances. For a specific time window, the complete multimodal observation for instance i is defined as $X_i = (X_i^M, X_i^L, X_i^T)$, where $X_i^M \in \mathbb{R}^{T \times d_m}$ represents continuous metric time series, X_i^L denotes semi-structured log sequences, and X_i^T captures distributed trace graphs. The system-wide input is aggregated as $X = \{X_i\}_{i=1}^N$.

To formalize missing modalities caused by infrastructure vulnerabilities, we define an observation mask vector $O = [o^M, o^L, o^T] \in \{0, 1\}^3$, where $o^k = 1$ indicates successful collection of modality $k \in \{M, L, T\}$, and $o^k = 0$ denotes its absence. The actual incomplete input is $\tilde{X} = \{\tilde{X}_i\}_{i=1}^N$, with each modality defined as $\tilde{X}_i^k = X_i^k$ if $o^k = 1$, and $\tilde{X}_i^k = \mathbf{0}$ (a structure-compatible static placeholder) otherwise.

Unlike existing methods [29, 40, 42] assuming $O = [1, 1, 1]$, we aim to learn a robust mapping $F_\theta : (\tilde{X}, O) \rightarrow (y, s, P)$ that explicitly leverages O to mitigate imputation noise. This function jointly optimizes three tasks: (1) AD, predicting a binary indicator $y \in \{0, 1\}$; (2) FT, classifying the failure type $s \in \mathcal{S}$ from a predefined taxonomy; and (3) RCL, estimating a probability distribution $P = [p_1, \dots, p_N] \in [0, 1]^N$ over instances. The core challenge is ensuring F_θ remains invariant to distributional shifts induced by static placeholders ($\mathbf{0}$) when $o^k = 0$, preserving the diagnostic fidelity of available modalities.

4 Methodology

4.1 Overview

As shown in Figure 5, ARMOR operates in two phases. During the *offline* phase, Module 1 first partitions the multimodal input and extracts modality-specific features independently for each modality from anomaly-free operational data, preserving the distinct semantics of continuous metrics and sparse events. Module 2 then integrates these isolated embeddings via a masked gating mechanism and a topology-aware graph network, establishing a normal operating baseline through self-supervised reconstruction so that subsequent anomalous deviations become identifiable. During the *online* phase, the trained model encodes incoming data to obtain latent embeddings alongside next-step predictions; the continuous deviations between predictions and observations are concatenated with the latent embeddings to form unified failure representations. These representations serve as the shared foundation for Module 3,

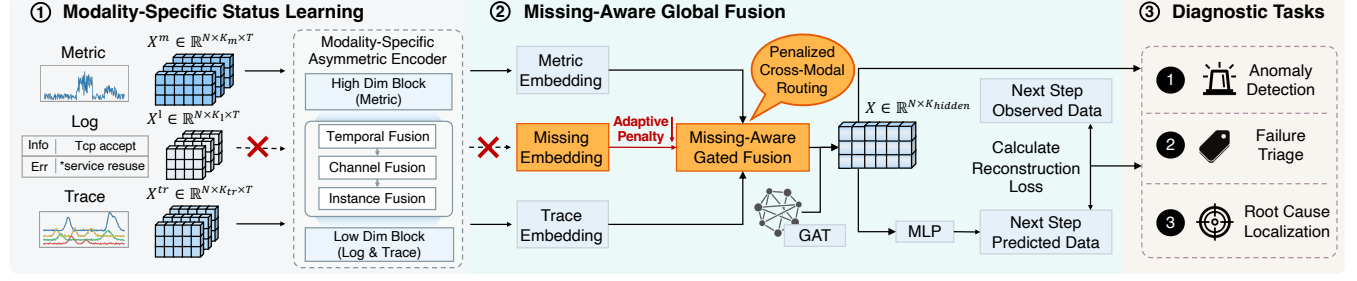


Figure 5: Overview of the proposed missing-aware incident management framework. It comprises three core modules: (1) Modality-Specific Status Learning, extracting intra-modality features via an asymmetric encoder; (2) Missing-Aware Global Fusion, integrating isolated embeddings through an attention-guided gating mechanism and a topology-aware graph network; and (3) Diagnostic Tasks, optimizing the network via offline self-supervised reconstruction and executing online AD, FT, and RCL using the unified representations.

which sequentially monitors for anomalies, determines the failure type upon detection, and localizes the root cause instance.

4.2 Modality-Specific Status Learning

Multimodal Data Serialization. Following established practices [17, 42], we transform metrics, logs, and traces into aligned time series. While metrics only require standard normalization, logs are parsed into event frequency series, and traces are aggregated into minute-level statistics (e.g., average latency, request counts). We concatenate these into a fused multivariate time series $M = [M_{metric} \parallel M_{log} \parallel M_{trace}]$ and apply nearest-neighbor interpolation to standardize the resolution to one minute. Finally, z-score standardization [1] over a sliding window produces the normalized input sequence $X = \{X^{(1)}, \dots, X^{(T)}\}$, where T denotes the window length, and each snapshot $X^{(t)} \in \mathbb{R}^{N \times K}$ encompasses N instances and K channels. This preliminary fusion effectively aligns heterogeneous data [15, 17, 39, 42, 54], leaving complex dependency modeling to the subsequent network.

Modality-Specific Feature Extraction. Directly processing the concatenated K channels overlooks the fundamental **structural asymmetry** of cloud-native telemetry: metrics reflect macroscopic, continuous resource states, whereas logs and traces record microscopic, discrete execution events. To address this, we partition the sequence into modality-specific sub-matrices X_M , X_L , and X_T . Let $X_m \in \mathbb{R}^{N \times K_m \times T}$ denote the input for modality m . We employ a *domain-guided asymmetric encoder* based on ModernTCN [12] to hierarchically disentangle dependencies. Large-kernel depthwise convolutions first capture long-term temporal trends (e.g., memory leaks):

$$H_{m,temp} = \text{DepthwiseConv}(X_m). \quad (1)$$

Pointwise convolutions then integrate intra-instance channel correlations (e.g., simultaneous latency and I/O spikes):

$$H_{m,chan} = \text{PointwiseConv}_{chan}(H_{m,temp}). \quad (2)$$

Finally, to facilitate early spatial interactions, we permute dimensions and apply instance-level convolution:

$$H_m = \text{PointwiseConv}_{inst}(\text{Permute}(H_{m,chan})). \quad (3)$$

Crucially, applying symmetric extractors to such heterogeneous inputs risks overfitting on sparse events or underfitting on volatile resources. Therefore, our asymmetric design configures high-capacity ModernTCN blocks for continuous metrics to capture intricate fluctuations, while employing lightweight blocks for sparse logs and traces. This physical alignment prevents infrastructure-level noise from overwhelming discrete event semantics. Consequently, the resulting embeddings retain distinct hidden dimensions: $H_M \in \mathbb{R}^{N \times d_M}$, $H_L \in \mathbb{R}^{N \times d_L}$, and $H_T \in \mathbb{R}^{N \times d_T}$, with $d_M > d_L, d_T$.

4.3 Missing-Aware Global Fusion

Constructing a comprehensive system representation from isolated unimodal embeddings requires addressing the frequent telemetry outages (e.g., agent crashes or network throttling) prevalent in production environments. Inspired by robust multimodal routing paradigms [28], we design a missing-aware global fusion module that employs an attention-guided gating mechanism for incomplete inputs and a graph neural network for spatial service dependencies.

We first project unimodal embeddings H_M, H_L, H_T to a unified dimension d , forming a token sequence $V = [v_M, v_L, v_T]$ per instance. A critical challenge arises when a modality collapses: replacing the missing high-dimensional tensor with static zero-padding induces a severe *distribution shift*, causing the network to misinterpret the absence of data as a "healthy idle" state (i.e., pseudo-normal noise). To explicitly isolate this infrastructure-induced artifact, we replace absent tokens ($o_k = 0$) with a *learnable placeholder embedding* $E_{miss} \in \mathbb{R}^d$. Unlike static zeros, E_{miss} acts as a structural *negative signal*, informing the fusion layer of telemetry loss rather than fabricating false stability. We further add a modality-specific embedding $E_{mod,k}$ to preserve semantics. The mask-adjusted token v'_k is formulated as:

$$v'_k = o_k v_k + (1 - o_k) E_{miss} + E_{mod,k}. \quad (4)$$

As shown in Figure 6, the adjusted sequence V' passes through a multi-head attention (MHA) layer to capture cross-modal interactions:

$$U = \text{LayerNorm}(V' + \text{Dropout}(\text{MHA}(V', V', V'))). \quad (5)$$

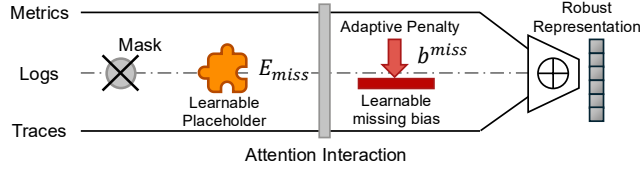


Figure 6: Missing-aware gated fusion. When a telemetry outage occurs (e.g., absent logs), the module first isolates the missing modality by injecting an explicit placeholder (E_{miss}). After cross-modal attention interaction, it applies a learnable negative bias ($b_{miss} < 0$) to suppress the routing contribution of the absent modality, ultimately producing a robust instance representation

We then dynamically calculate a gating weight for each enriched modality token U_k . In a highly dynamic microservice architecture, relying equally on a generalized placeholder and actual telemetry data can derail root cause localization. Since the placeholder lacks the precise execution context of the missing modality, the network must adaptively downgrade its routing contribution and redirect diagnostic attention toward the surviving, reliable signals from other modalities. To enforce this adaptive penalization during cross-modal routing, we compute the gating logit s_k by applying a shared linear projection $W_s \in \mathbb{R}^{1 \times d}$ and adding a learnable missing bias $b_k^{miss} \in \mathbb{R}_{<0}$ (constrained to be negative) exclusively when $o_k = 0$:

$$s_k = W_s U_k + (1 - o_k) b_k^{miss}. \quad (6)$$

The fusion weights α_k are obtained via a softmax function over the logits. Subsequently, the robust instance representation $E \in \mathbb{R}^{N \times d}$ is computed as the weighted sum of the enriched tokens across all modalities, where N represents the total number of microservice instances:

$$E = \sum_{k \in \{M, L, T\}} \alpha_k U_k. \quad (7)$$

Microservice systems operate on complex spatial topologies defined by invocation dependencies. To capture these structural interactions, we model the system as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ represents the microservice instances and $\mathcal{E}$ denotes the interaction links. We apply a multi-layer graph attention network [29, 45] to propagate the fused features E along this behavioral graph. Let $\mathcal{N}(i)$ denote the set of topological neighbors for instance i , including the instance itself. The graph convolution aggregates spatial features by computing attention coefficients α_{ij}^c that quantify the influence of neighbor j on instance i . To process information from multiple representation subspaces, we configure the network with a multi-head attention mechanism. The feature update for instance i at the $(l+1)$ -th layer is formulated as:

$$Z_i^{l+1} = \text{ELU} \left(\frac{1}{C} \sum_{c=1}^C \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^c W^c Z_j^l \right), \quad (8)$$

where Z_i^l represents the latent feature of instance i at the l -th layer, and the initial node feature is set to the fused representation $Z^0 = E$. The parameter C specifies the number of independent attention heads, while W^c denotes the linear projection matrix for head c . To

prevent overfitting during spatial propagation, dropout operations are applied to both the feature matrices and the attention coefficients. Finally, the exponential linear unit [9] activation function is applied after averaging the aggregated outputs from all attention heads.

4.4 Offline Model Optimization

Although Figure 5 primarily illustrates the online diagnostic pipeline, the effectiveness of the unified representations depends entirely on the offline training phase. During this stage, we optimize the entire framework via a self-supervised auto-regression task. To simulate infrastructure-level outages, we apply stochastic modality dropout augmentation: during each training iteration, the binary observation mask O is generated by independently dropping entire modality channels, forcing the framework to infer missing operational states from the surviving signals of topologically correlated microservices. A multi-layer perceptron projects the topology-aware representation $Z^{(T)}$ to predict the full-channel data of the next time step: $\hat{X}^{(T+1)} = \text{MLP}(Z^{(T)})$. By reconstructing both observed and masked modalities, the model learns cross-modal invariants (e.g., inferring CPU trends from trace volumes). The training objective comprises two masked mean squared error terms:

$$\mathcal{L}_{obs} = \frac{1}{\sum O} \left\| (\hat{X}^{(T+1)} - X^{(T+1)}) \odot O \right\|_F^2, \quad (9)$$

$$\mathcal{L}_{miss} = \frac{1}{\sum (1-O)} \left\| (\hat{X}^{(T+1)} - X^{(T+1)}) \odot (1-O) \right\|_F^2. \quad (10)$$

Note that $\mathcal{L}_{miss}$ is computable during training because the offline phase uses complete historical archives; missing channels are produced by stochastic dropout augmentation, not structural absence. The total objective is $\mathcal{L} = \mathcal{L}_{obs} + \lambda \mathcal{L}_{miss}$, with λ serving as a balancing weight. For downstream diagnostic tasks, we construct a unified failure signature by concatenating reconstruction errors and latent embeddings as $R^{(t)} = [(\hat{X}^{(t)} - X^{(t)}) \parallel Z^{(t)}]$, where $\parallel$ denotes concatenation.

4.5 Online Diagnostic Tasks

During online execution, unified representations drive three sequential diagnostic tasks. AD and RCL require no manual annotations; FT relies solely on failure-type labels for its downstream classifier.

Anomaly Detection. Explicit reconstruction errors often miss subtle deviations in latent topological states. Therefore, we project the unified representation $R^{(t)}$ at each time step t into a scalar system deviation score $S^{(t)}$ to improve sensitivity. Instead of relying on predefined distribution assumptions, we dynamically calculate the detection threshold using the streaming peaks-over-threshold method based on extreme value theory [34], fitted to normal operating scores [25, 26, 54]. An anomaly is flagged whenever $S^{(t)}$ exceeds this threshold. To distinguish actual system failures from benign fluctuations (e.g., workload spikes), the system generates a formal alert ($y = 1$) and routes it to the triage module only if the anomalous state persists across a delay window W_d .

Failure Triage. By keeping the upstream encoder frozen, SREs can retrain the failure classifier for new fault categories without retraining the entire model. Prior studies [42] typically employ naive mean pooling, which smooths out critical transient spikes. To assess the failure category $s \in \{1, \dots, K\}$, we apply

the reconstruction error component $R_{err}^{(t)} = \hat{X}^{(t)} - X^{(t)}$ and construct a system-level representation via statistical pooling: $R_{sys} = \text{Concat}(\text{Mean}(R_{err}), \text{Max}(R_{err}), \text{Std}(R_{err}))$, capturing baseline deviations, peak severities, and volatility. We feed R_{sys} into a light-weight XGBoost [7] classifier with sample weighting to handle class imbalance.

Root Cause Localization. Traditional techniques ignoring latent topological semantics are insufficient. Our approach uses the full unified representation $R^{(t)}$ and compares instance-level signatures against the time-aggregated global failure vector $\bar{R}_{sys} = \text{Mean}(\{R^{(t)}\}_{t=1}^{W_f})$ via cosine similarity across the failure window of length W_f :

$$p_i = \frac{1}{W_f} \sum_{t=1}^{W_f} \frac{R_i^{(t)} \cdot \bar{R}_{sys}}{\|R_i^{(t)}\| \|\bar{R}_{sys}\|}. \quad (11)$$

This computation yields a ranked probability distribution $P = [p_1, \dots, p_N]$ across all instances, where higher similarity indicates a greater probability of being the root cause.

5 Evaluation

In this section, we address the following research questions:

- (1) **RQ1:** How well does ARMOR perform in AD, FT, and RCL?
- (2) **RQ2:** How robust is ARMOR under missing modalities?
- (3) **RQ3:** Does each core component contribute to ARMOR?
- (4) **RQ4:** How do the major hyperparameters influence the performance of ARMOR?

5.1 Experimental Setup

5.1.1 Datasets. To maintain fairness with existing methods and ensure comprehensive benchmarking, we evaluate ARMOR on two industry-standard datasets ($\mathcal{D}_1$ and $\mathcal{D}_2$) representing distinct microservice architectures. Both datasets encompass continuous metrics, discrete logs, and distributed traces, with each incident containing ground-truth annotations for the timestamp, failure type, and root cause instance.

$\mathcal{D}_1$ originates from a cloud-deployed e-commerce simulation replaying authentic historical incidents. It features a 46-node architecture (40 microservices and 6 virtual machines) with 5 distinct system-level failure categories (e.g., CPU, memory, and network faults). The dataset comprises 3,714 normal periods and 210 failure cases. Its extensive multimodal records include 44,858,388 traces, 66,648,685 logs, and 20,917,746 metrics [42].

$\mathcal{D}_2$ originates from the International AIOps Challenge 2021¹, representing the core management system of a top-tier commercial bank. It is composed of 18 heterogeneous instances spanning six months of operation. The dataset includes 12,297 normal periods and 133 failure cases across 6 distinct failure types (e.g., JVM and node-level anomalies). Its massive telemetry records comprise 214,337,882 traces, 21,356,870 logs, and 12,871,809 metrics.

Following established chronological evaluation practices [15, 27, 42], we split each dataset using the first timestamp in the last 40% of failure cases as the cutoff point. This ensures temporal continuity between the training (first 60%) and testing (remaining 40%) phases.

¹<https://aiops-challenge.com>

5.1.2 Baseline Approaches. We compare ARMOR against nine state-of-the-art baselines, which include multi-task frameworks and single-task methods. The multi-task frameworks consist of ART [42], TrioXpert [40], Eadro [17], DiagFusion [51], and DejaVu [20]. For single-task methods, we select Hades [18] and ChronoSage [52] for AD, MicroCBR [23] for FT, and DeepHunt [39] for RCL. The baselines are configured according to the original papers, with dataset-specific adjustments (e.g., window length). To ensure an independent evaluation, we assume known incident timestamps when assessing FT and RCL for the methods that lack AD modules [42].

5.1.3 Simulation of Modality Collapse. As discussed in Section 2.1, microservice systems experience infrastructure-level modality collapse (e.g., agent crashes), losing entire observability streams. We therefore simulate complete modality missingness rather than fine-grained noise. During training, stochastic modality dropout augmentation randomly drops entire modalities. During evaluation, a binary vector $O \in \{0, 1\}^3$ systematically masks specific modalities; e.g., $O = [0, 1, 1]$ zeroes out all metric channels, mirroring the Prometheus blackout in Figure 2.

5.1.4 Evaluation Metrics. We formulate AD and FT as classification tasks. AD is a binary classification problem that identifies whether a failure occurred, whereas FT is a multi-class classification problem that determines the specific failure type. We calculate the precision, recall, and F1-score based on true positive, false positive, and false negative samples. Specifically, these metrics are formulated as Precision = $TP / (TP + FP)$, Recall = $TP / (TP + FN)$, and F1 = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$. To handle the imbalanced distribution of the failure types in FT, we report the weighted average F1-score. For RCL, we evaluate the ranking accuracy of the culprit instances by using the Top@K metric, which is defined as $\text{Top}@K = \frac{1}{N} \sum_{i=1}^N (gt_i \in P_{i[1:K]})$. Here, N denotes the total number of evaluated failures, gt_i represents the ground-truth root cause for the i -th failure case, and $P_{i[1:K]}$ denotes the top- K predicted candidates. We also compute the average score across the top five results, which is formulated as $\text{Avg}@5 = \frac{1}{5} \sum_{K=1}^5 \text{Top}@K$.

5.1.5 Implementation. We implement ARMOR and all the baseline methods by using Python 3.9.13, PyTorch 1.12.1 (CUDA 11.6), and DGL 0.9.0 (CUDA 11.6). All the experiments are conducted on a dedicated server equipped with a 20-vCPU Intel Xeon Platinum 8470Q processor and a single NVIDIA RTX 4090 GPU (24GB).

5.2 RQ1: Overall Performance

As Table 1 shows, ARMOR consistently achieves the highest performance across all tasks on both datasets. Single-task methods (e.g., Hades, ChronoSage, MicroCBR, DeepHunt) are competitive on individual tasks but ignore shared contextual semantics, exacerbating cascading errors. Supervised multi-task frameworks (e.g., Eadro, DiagFusion, DejaVu) rely heavily on historical fault labels, limiting generalization. Self-supervised frameworks (e.g., ART, TrioXpert) reduce label dependency but employ rigid fusion paradigms that fail to accommodate the structural asymmetry between dense continuous metrics and sparse discrete events. In contrast, ARMOR's

Table 1: Overall performance comparison of AD, FT, and RCL on datasets $\mathcal{D}_1$ and $\mathcal{D}_2$ under complete data conditions. The best results are highlighted in bold. Hyphens indicate that the method does not support the corresponding task.

Type	Method	$\mathcal{D}_1$									$\mathcal{D}_2$								
		AD			FT			RCL			AD			FT			RCL		
		Precision	Recall	F1	Precision	Recall	F1	Top1	Top3	AVG@5	Precision	Recall	F1	Precision	Recall	F1	Top1	Top3	AVG@5
Multiple	ARMOR	0.925	1.0	0.961	0.946	0.941	0.938	0.821	0.941	0.910	0.993	1.0	0.997	0.882	0.870	0.869	0.815	0.907	0.893
	ART [42]	0.899	0.990	0.942	0.836	0.809	0.812	0.667	0.810	0.776	0.877	0.960	0.917	0.851	0.796	0.802	0.722	0.889	0.870
	TrioXpert [40]	0.880	0.972	0.924	0.852	0.768	0.807	0.651	0.778	0.773	0.854	0.972	0.909	0.814	0.725	0.767	0.550	0.775	0.750
	Eadro [17]	0.425	0.946	0.586	-	-	-	0.137	0.315	0.302	0.767	0.935	0.842	-	-	-	0.157	0.315	0.310
	DiagFusion [51]	-	-	-	0.675	0.500	0.568	0.310	0.452	0.467	-	-	-	0.797	0.527	0.593	0.582	0.709	0.695
	Dejavu [20]	-	-	-	0.369	0.621	0.415	0.411	0.679	0.625	-	-	-	0.718	0.340	0.417	0.402	0.667	0.619
Single	Hades [18]	0.866	0.863	0.865	-	-	-	-	-	-	0.867	0.868	0.868	-	-	-	-	-	-
	ChronoSage [52]	0.904	0.842	0.872	-	-	-	-	-	-	0.945	1.0	0.972	-	-	-	-	-	-
	MicroCBR [23]	-	-	-	0.667	0.796	0.717	-	-	-	-	-	-	0.629	0.678	0.636	-	-	-
	DeepHunt [39]	-	-	-	-	-	-	0.796	0.905	0.897	-	-	-	-	-	-	0.731	0.893	0.873

domain-guided asymmetric encoder and self-supervised reconstruction are aligned with the physical reality of microservice telemetry, effectively mitigating negative transfer across tasks. Across all methods and both datasets, ARMOR achieves the best results on every metric. Particularly in the Failure Triage (FT) task, ARMOR improves over the overall best-performing baseline by 15.5% in F1-score on $\mathcal{D}_1$ and 8.4% on $\mathcal{D}_2$. Furthermore, in Root Cause Localization (RCL), it surpasses the strongest multi-task baseline by up to 17.3% in Avg@5 on $\mathcal{D}_1$.

Beyond diagnostic accuracy, Table 2 shows that ARMOR consistently achieves lower latency than ART across all tasks on both datasets, confirming that the shared encoder and parallelized fusion introduce minimal overhead for online deployment.

5.3 RQ2: Robustness under Missing Modalities

To evaluate framework robustness against incomplete data, we simulate vulnerabilities in the observability infrastructure by systematically masking specific modality combinations during the online inference phase. We design six distinct missing scenarios that encompass single-modality and dual-modality absences. We focus on ART as the robustness reference: it shares ARMOR’s self-supervised orientation and unified task scope, making it the most meaningful counterpart. Supervised baselines require fault labels incompatible with our protocol; single-task methods cover only pipeline subsets. Figure 7a compares ARMOR against ART. To ensure a fair comparison, we retrain ART under the same modality dropout augmentation protocol applied to ARMOR, additionally equipping it with three standard imputation strategies (zero-padding, mean imputation, and KNN imputation), and report its best result per scenario. Hatched regions denote the performance gap. ARMOR outperforms ART across all scenarios on both datasets.

The performance degradation exhibits a distinct asymmetry depending on the missing modalities. While the absence of sparse events (such as logs or traces) causes relatively constrained declines, the absence of continuous metrics, which encode critical system states, triggers a severe systemic collapse in the baseline model. Across all varying degrees of data corruption, ARMOR consistently maintains a highly stable functional baseline. The performance gap between the two models widens substantially as more modalities are removed: when the metric modality is absent, ARMOR retains over 60% higher RCL accuracy than ART across both datasets. This empirical superiority directly validates the proposed missing-aware

Table 2: Total execution time (s) of the diagnostic pipeline (AD, FT, and RCL) over the entire test set on $\mathcal{D}_1$ and $\mathcal{D}_2$.

Method	$\mathcal{D}_1$			$\mathcal{D}_2$		
	AD	FT	RCL	AD	FT	RCL
ART [42]	6.81	5.05	22.46	28.81	4.01	12.91
ARMOR	5.23	1.56	21.52	6.71	1.45	8.08

global fusion mechanism. Because ART is rigidly designed under the assumption of complete data, encountering missing modalities forces the use of naive static padding to maintain input dimensions. This passive imputation introduces pseudo-normal noise that distorts the surviving healthy signals. In contrast, ARMOR inherently accommodates missing data by using learnable placeholders and dynamic gating biases to adaptively redistribute routing attention and isolate the semantic impact of missing data.

Figure 7b visualizes the learned dynamic bias b_k^{miss} for each missing modality. Two observations validate our design: (1) all learned biases are strictly negative, confirming that the network actively penalizes placeholder tokens to suppress their routing contributions; (2) the penalty magnitude aligns with structural asymmetry: the model learns a significantly larger negative bias for missing metrics than for sparser logs and traces, as metric placeholders introduce the most severe pseudo-normal noise. This confirms that ARMOR automatically quantifies and mitigates the varying degrees of semantic threat posed by different modality failures.

5.4 RQ3: Ablation Study

To evaluate the key technical contributions of ARMOR, we design twelve variants grouped into two categories. The first category assesses the modality-specific status learning and missing-aware global fusion. Specifically, variants A1, A2, and A3 remove the temporal, channel, and instance fusions, respectively. Variant A4 replaces the missing-aware gated fusion with a naive mean fusion. Variant A5 replaces the learnable missing placeholder with static zero-padding. Variant A6 trains the model exclusively on complete modalities, removing missing modality augmentation. Variants A7, A8, and A9 modify the graph network by removing it entirely, replacing it with GCN, or replacing it with GraphSAGE, respectively. The second category evaluates the unified representation strategies

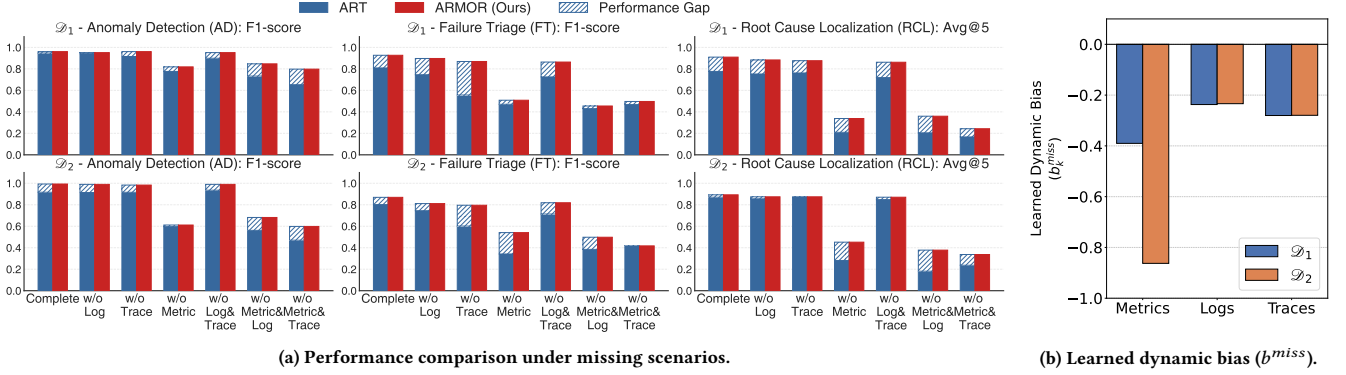


Figure 7: Robustness analysis against missing modalities. (a) Performance comparison across AD, FT, and RCL under various missing modality scenarios on datasets $\mathcal{D}_1$ and $\mathcal{D}_2$. The solid bars represent the diagnostic accuracy of ARMOR and ART, while the hatched regions denote the explicit performance gap indicating the margin by which ART falls behind ARMOR. (b) Visualization of the learned dynamic bias (b^{miss}) for missing modalities. The model actively learns negative biases to suppress the routing contribution of absent data, with the penalty being most severe for the high-dimensional metrics.

Table 3: Ablation study on datasets $\mathcal{D}_1$ and $\mathcal{D}_2$. Variants A1-A9 evaluate the encoder and global fusion components. Variants B1-B3 evaluate task-specific unified representation strategies. Hyphens indicate that the variant evaluates a specific downstream task and remains identical to the base model for the other tasks. The best results are highlighted in bold.

Variant	$\mathcal{D}_1$			$\mathcal{D}_2$		
	AD (F1)	FT (F1)	RCL (Avg@5)	AD (F1)	FT (F1)	RCL (Avg@5)
A1	0.956	0.897	0.902	0.983	0.843	0.874
A2	0.937	0.915	0.883	0.980	0.787	0.885
A3	0.937	0.914	0.891	0.987	0.861	0.870
A4	0.942	0.926	0.900	0.990	0.794	0.878
A5	0.939	0.887	0.875	0.964	0.771	0.856
A6	0.946	0.883	0.860	0.941	0.822	0.885
A7	0.960	0.895	0.857	0.973	0.831	0.867
A8	0.946	0.895	0.829	0.959	0.859	0.882
A9	0.933	0.927	0.860	0.987	0.865	0.889
B1	0.956	-	-	0.964	-	-
B2	-	0.908	-	-	0.830	-
B3	-	-	0.767	-	-	0.889
ARMOR	0.961	0.938	0.910	0.997	0.869	0.893

for downstream tasks. In this category, B1 and B3 remove the latent embeddings for AD and RCL, respectively, whereas B2 removes the standard deviation feature for FT.

Table 3 shows that ARMOR outperforms all variants on both datasets. Performance drops when removing the temporal, channel, or instance fusions (A1–A3), confirming that hierarchical extraction is essential for capturing complex telemetry dynamics. The decline in A4 proves that naive mean fusion fails to suppress imputation noise, validating the gating mechanism. The drops in A5 and A6 confirm our two-pronged missing modality strategy: static zero-padding misleads the network with pseudo-normal signals (A5), and training without modality dropout prevents robust representation learning (A6). Replacing or removing the graph network (A7–A9)

degrades localization, indicating that attention-guided spatial propagation better models heterogeneous invocation dependencies than convolutional aggregators. Modifying the task-specific representations (B1–B3) consistently reduces performance, confirming that reconstruction errors, deviation volatility, and topology-encoded embeddings each provide indispensable diagnostic signals for their respective tasks.

5.5 RQ4: Hyperparameter Sensitivity

As Figure 8 illustrates, for High Dim Blocks and GAT, performance peaks at moderate depth and degrades beyond it due to overfitting and over-smoothing; Low Dim Blocks are optimal at depth 1, reflecting the simpler structure of sparse modalities. This asymmetry is consistent with the structural difference between dense continuous metrics and sparse discrete logs and traces, where deeper extraction benefits the former but hurts the latter. Window size T follows the same trend: too small misses failure dynamics, too large introduces noise, with $T = 6$ providing the best balance. These results confirm that ARMOR is robust to reasonable hyperparameter choices without dataset-specific tuning.

6 Discussion

6.1 Limitations and Possible Solutions

ARMOR’s offline training assumes that historical records are structurally complete enough to serve as reconstruction targets for $\mathcal{L}_{miss}$. In the rare case where a modality has been persistently absent from the archive to an extent beyond the stochastic dropout simulated during training, the missing-aware fusion module loses its explicit training signal. Replacing strict numerical reconstruction with cross-instance contrastive learning, aligning incomplete instances against fully observed peers, offers a viable path forward without requiring perfect historical targets. A second limitation

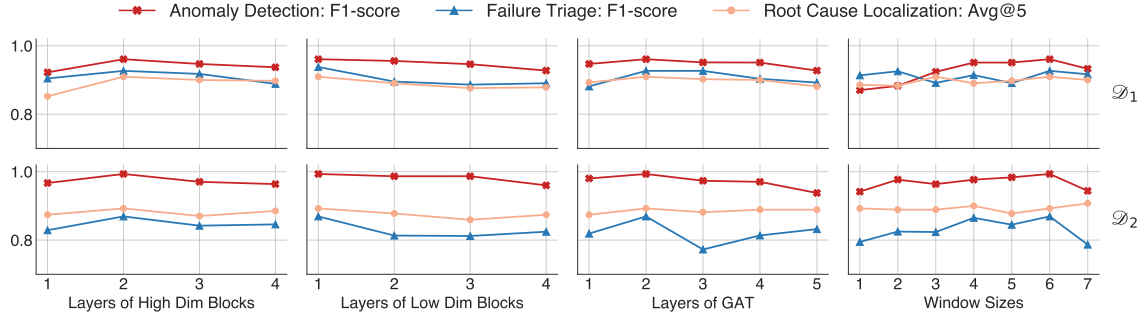


Figure 8: Sensitivity analysis of four core hyperparameters (Layers of High Dim Blocks, Layers of Low Dim Blocks, Layers of GAT, and Window Sizes) across AD, FT, and RCL on datasets $\mathcal{D}_1$ and $\mathcal{D}_2$.

is topology evolution: the dependency graph is built from historical records, so frequent instance creation or destruction requires periodic updates; incremental graph learning is a natural extension.

6.2 Threats to Validity

One threat concerns construct validity. Our robustness experiments mask entire modality channels, directly mirroring the complete-stream blackouts in Section 2.1: process crashes and network blocks eliminate an entire channel at once. Sub-channel partial loss is a strictly less severe case for ARMOR’s gating mechanism; finer-grained evaluation is left to future work. Furthermore, both benchmark datasets provide complete multimodal archives for the normal training period, so evaluating training-time structural missingness is similarly deferred to future work. A second threat concerns internal validity: to rule out imputation bias, we retrain ART under the same modality dropout augmentation protocol as ARMOR, additionally equipping it with zero-padding, mean imputation, and KNN imputation, and report its best result per scenario; the observed gap therefore reflects architectural differences rather than training asymmetry. A third threat concerns external validity: both datasets originate from controlled testbed environments with fewer instances than large-scale industrial deployments and may not capture cascading or timing-sensitive failures. Extending evaluation to live industrial traces remains important future work.

7 Related Work

Single-Task Incident Diagnosis. Early approaches address individual incident management stages, such as AD [4–6, 8, 11, 15, 16, 18, 52], FT [24, 37, 38, 44], and RCL [14, 22, 32, 35, 39], in isolation. Although effective locally, isolating these related tasks ignores shared diagnostic contexts, increases maintenance overhead, and renders independent pipelines vulnerable to severe error cascading [42].

Unified Multi-Task Frameworks. Recent multi-task approaches extract shared knowledge from multimodal data (metrics, logs, and traces) to construct unified frameworks that overcome these limitations. Supervised multi-task models [10, 17] integrate heterogeneous events but require extensive fault-labeled data throughout training, limiting practical deployment in environments where labeled fault data is prohibitively expensive. To eliminate label

dependency, self-supervised unified approaches [29, 40, 42] employ cascaded representation learning to jointly optimize AD, FT, and RCL. However, existing self-supervised unified frameworks assume perfectly complete observability data, leaving them without a principled missing-aware strategy for production deployment. ARMOR addresses this gap by combining the label-free orientation of self-supervised frameworks with explicit architectural support for modality absence.

Missing Modality Learning. Handling incomplete multimodal inputs has been studied in general settings through generative imputation [46, 49] and representation-level routing [19, 21, 28, 47, 48]. However, these methods either introduce substantial computational overhead from data synthesis or rely on a primary modality for alignment, which fails when the most information-dense stream drops. More critically, microservice observability data exhibits structural asymmetry between dense continuous metrics and sparse discrete logs and traces, a property that general missing modality methods do not account for. To our knowledge, ARMOR is the first self-supervised framework to address missing modalities in the unified incident management setting, treating observability gaps as a first-class design concern rather than a post-hoc fallback that requires fault-labeled training data.

8 Conclusion

This paper presents ARMOR, a robust self-supervised framework for automated incident management under missing modalities. ARMOR employs a modality-specific asymmetric encoder and a missing-aware gated fusion mechanism with learnable placeholders and dynamic bias compensation to isolate imputation noise. The unified failure representations concurrently support label-free AD and RCL, with FT requiring only failure-type annotations for a lightweight downstream classifier. Extensive evaluations demonstrate state-of-the-art performance under complete data and strong robustness under severe modality loss. Future work will investigate cross-instance contrastive learning to reduce reliance on complete historical archives.

9 Data Availability Statement

The source code and data of ARMOR is available in Zenodo [2].

References

- [1] Luai Al Shalabi, Ziyad Shaaban, and Basel Kasasbeh. 2006. Data mining: A preprocessing engine. *Journal of Computer Science* 2, 9 (2006), 735–739.
- [2] ARMOR. 2026. . <https://doi.org/10.5281/zenodo.19157589>
- [3] Sachin Ashok, Vipul Harsh, Brighten Godfrey, Radhika Mittal, Srinivasan Parthasarathy, and Larisa Shwartz. 2024. Traceweaver: Distributed request tracing for microservices without application modification. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 828–842.
- [4] Julien Audibert, Pietro Michiardi, Frédéric Guyard, Sébastien Marti, and Maria A Zuluaga. 2020. Usad: Unsupervised anomaly detection on multivariate time series. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 3395–3404.
- [5] Feiyi Chen, Zhen Qin, Mengchu Zhou, Yingying Zhang, Shuiguang Deng, Lunting Fan, Guansong Pang, and Qingsong Wen. 2024. Lara: A light and anti-overfitting retraining approach for unsupervised time series anomaly detection. In *Proceedings of the ACM Web Conference 2024*. 4138–4149.
- [6] Feiyi Chen, Yingying Zhang, Lunting Fan, Yuxuan Liang, Guansong Pang, Qingsong Wen, and Shuiguang Deng. 2024. Cluster-wide task slowdown detection in cloud system. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 266–277.
- [7] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [8] Dongchan Cho, Jiho Han, Keumyeong Kang, Minsang Kim, Honggyu Ryu, and Namsoon Jung. 2025. Structured Temporal Causality for Interpretable Multivariate Time Series Anomaly Detection. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=V5kzCSeAXF>
- [9] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. 2015. Fast and accurate deep network learning by exponential linear units (elus). *arXiv preprint arXiv:1511.07289* 4, 5 (2015), 11.
- [10] Kaiqi Ding, Yuanmu Ma, and Kaigui Bian. 2025. Adaptive Modality Compensation via Bi-Mamba Dual-Stream Learning for Microservice Failure Diagnosis Under Incomplete Multimodal Data. In *International Conference on Service-Oriented Computing*. Springer, 171–188.
- [11] Kaiqi Ding, Yuanmu Ma, Zijian Song, and Kaigui Bian. 2025. Enhancing Microservices Anomaly Detection via Multimodal Data Fusion in the Wavelet Domain and Spatiotemporal Graph-based Diffusion Probabilistic Model. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 510–520.
- [12] Luo donghao and wang xue. 2024. ModernTCN: A Modern Pure Convolution Structure for General Time Series Analysis. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=vpJMjXHU>
- [13] Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina. 2017. Microservices: yesterday, today, and tomorrow. *Present and ulterior software engineering* (2017), 195–216.
- [14] Xiao Han, Saima Absar, Lu Zhang, and Shuhan Yuan. 2025. Root Cause Analysis of Anomalies in Multivariate Time Series through Granger Causal Discovery. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=k38Th3x4d9>
- [15] Jun Huang, Yang Yang, Hang Yu, Jianguo Li, and Xiao Zheng. 2023. Twin graph-based anomaly detection via attentive multi-modal learning for microservice system. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 66–78.
- [16] Hyungi Kim, Jisoo Mok, Dongjun Lee, Jaihyun Lew, Sungjae Kim, and Sungroh Yoon. 2025. Causality-Aware Contrastive Learning for Robust Multivariate Time-Series Anomaly Detection. In *International Conference on Machine Learning*. PMLR, 30591–30608.
- [17] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R Lyu. 2023. Eadro: An end-to-end troubleshooting framework for microservices on multi-source data. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1750–1762.
- [18] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, Yongqiang Yang, and Michael R Lyu. 2023. Heterogeneous anomaly detection for software systems via semi-supervised cross-modal attention. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1724–1736.
- [19] Sijie Li, Chen Chen, and Jungong Han. 2025. Simlml: A simple framework for multi-modal learning with missing modality. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 24068–24077.
- [20] Zeyan Li, Nengwen Zhao, Mingjie Li, Xianglin Lu, Lixin Wang, Dongdong Chang, Xiaohui Nie, Li Cao, Wenchu Zhang, Kaixin Sui, et al. 2022. Actionable and interpretable fault localization for recurring failures in online service systems. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 996–1008.
- [21] Ziyi Li, Wei-Long Zheng, and Bao-Liang Lu. 2025. Multimodal emotion recognition with missing modality via a unified multi-task pre-training framework. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 5717–5725.
- [22] Cheng-Ming Lin, Ching Chang, Wei-Yao Wang, Kuang-Da Wang, and Wen-Chih Peng. 2024. Root cause analysis in microservice using neural granger causal discovery. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 206–213.
- [23] Fengrui Liu, Yang Wang, Zhenyu Li, Rui Ren, Hongtao Guan, Xian Yu, Xiaofan Chen, and Gaogang Xie. 2022. Microbr: Case-based reasoning on spatio-temporal fault knowledge graph for microservices troubleshooting. In *International conference on case-based reasoning*. Springer, 224–239.
- [24] Minghua Ma, Zheng Yin, Shenglin Zhang, Sheng Wang, Christopher Zheng, Xinhao Jiang, Hanwen Hu, Cheng Luo, Yilin Li, Nengjun Qiu, et al. 2020. Diagnosing root causes of intermittent slow queries in cloud databases. *Proceedings of the VLDB Endowment* 13, 8 (2020), 1176–1189.
- [25] Minghua Ma, Shenglin Zhang, Junjie Chen, Jim Xu, Haozhe Li, Yongliang Lin, Xiaohui Nie, Bo Zhou, Yong Wang, and Dan Pei. 2021. Jump-Starting multivariate time series anomaly detection for online service systems. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 413–426.
- [26] Minghua Ma, Shenglin Zhang, Dan Pei, Xin Huang, and Hongwei Dai. 2018. Robust and rapid adaption for concept drift in software system anomaly detection. In *2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 13–24.
- [27] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, et al. 2019. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs.. In *IJCAI*, Vol. 19. 4739–4745.
- [28] Payal Mohapatra, Yueyuan Sui, Akash Pandey, Stephen Xia, and Qi Zhu. 2025. MAESTRO: Adaptive Sparse Attention and Robust Learning for Multimodal Dynamic Time Series. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=1K28gV5MeF>
- [29] Xiaohui Nie, Hang Cui, Changhua Pei, Haotian Si, Ke Xiang, Jingjing Li, Yanbiao Li, Gaogang Xie, and Dan Pei. 2025. DeST: An Unsupervised Decoupled Spatio-Temporal Framework for Microservice Incident Management. In *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 335–346.
- [30] Parametrix. 2023. *Cloud Outage and the Fortune 500*. Retrieved February 28, 2026 from <https://www.parametrixinsurance.com/reports-white-papers/cloud-outage-and-the-fortune-500>
- [31] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A Efros. 2016. Context encoders: Feature learning by inpainting. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2536–2544.
- [32] Luan Pham, Huong Ha, and Hongyu Zhang. 2024. Root cause analysis for microservice system based on causal inference: How far are we?. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 706–715.
- [33] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. 1986. Learning representations by back-propagating errors. *nature* 323, 6088 (1986), 533–536.
- [34] Alban Siffer, Pierre-Alain Fouque, Alexandre Termier, and Christine Largouet. 2017. Anomaly detection in streams with extreme value theory. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 1067–1075.
- [35] Gagan Somashekar, Anurag Dutt, Mainak Adak, Tania Lorido Botran, and Anshul Gandhi. 2024. GAMMA: Graph Neural Network-Based Multi-Bottleneck Localization for Microservices Applications. In *Proceedings of the ACM Web Conference 2024*. 3085–3095.
- [36] Akshitha Sriraman, Abhishek Dhanotia, and Thomas F Wenisch. 2019. Softsku: Optimizing server architectures for microservice diversity@ scale. In *Proceedings of the 46th International Symposium on Computer Architecture*. 513–526.
- [37] Yicheng Sui, Yuzhe Zhang, Jianjun Sun, Ting Xu, Shenglin Zhang, Zhengdan Li, Yongqian Sun, Fangrui Guo, Junyu Shen, Yuzhi Zhang, et al. 2023. Logkg: Log failure diagnosis through knowledge graph. *IEEE Transactions on Services Computing* 16, 5 (2023), 3493–3507.
- [38] Wu Sun, Panfeng Chen, Mei Chen, Hui Li, Yanhao Wang, Gang Huang, and Hongyuan Li. 2025. Failure Classification for Microservice Systems Based on Variational Graph Auto-Encoders. In *International Conference on Service-Oriented Computing*. Springer, 189–204.
- [39] Yongqian Sun, Zihan Lin, Binpeng Shi, Shenglin Zhang, Shiyu Ma, Pengxiang Jin, Zhenyu Zhong, Lemeng Pan, Yicheng Guo, and Dan Pei. 2025. Interpretable failure localization for microservice systems based on graph autoencoder. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–28.
- [40] Yongqian Sun, Yu Luo, Xidao Wen, Yuan Yuan, Xiaohui Nie, Shenglin Zhang, Tong Liu, and Xi Luo. 2025. TrioXpert: An Automated Incident Management Framework for Microservice System. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 3239–3250.
- [41] Yongqian Sun, Xijie Pan, Xiao Xiong, Lei Tao, Jiaju Wang, Shenglin Zhang, Yuan Yuan, Yuqi Li, and Kunlin Jian. 2025. ClusterRCA: An End-to-End Approach for Network Fault Localization and Classification for HPC System. In *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 347–358.

- [42] Yongqian Sun, Binpeng Shi, Mingyu Mao, Minghua Ma, Sibao Xia, Shenglin Zhang, and Dan Pei. 2024. Art: A unified unsupervised framework for incident management in microservice systems. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1183–1194.
- [43] Lei Tao, Xianglin Lu, Shenglin Zhang, Jiaqi Luan, Yingke Li, Mingjie Li, Zeyan Li, Qingyang Yu, Hucheng Xie, Ruijie Xu, et al. 2024. Diagnosing performance issues for large-scale microservice systems with heterogeneous graph. *IEEE Transactions on Services Computing* 17, 5 (2024), 2223–2235.
- [44] Lei Tao, Shenglin Zhang, Zedong Jia, Jinrui Sun, Minghua Ma, Zhengdan Li, Yongqian Sun, Canqun Yang, Yuzhi Zhang, and Dan Pei. 2024. Giving every modality a voice in microservice failure diagnosis via multimodal adaptive optimization. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1107–1119.
- [45] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=rJXMpikCZ>
- [46] Hu Wang, Yuanhong Chen, Congbo Ma, Jodie Avery, Louise Hull, and Gustavo Carneiro. 2023. Multi-modal learning with missing modality via shared-specific feature modelling. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 15878–15887.
- [47] Renjie Wu, Hu Wang, Hsiang-Ting Chen, and Gustavo Carneiro. 2026. Deep Multimodal Learning with Missing Modality: A Survey. *Transactions on Machine Learning Research* (2026). <https://openreview.net/forum?id=tc7RFcx4hT> Survey Certification.
- [48] Wenxin Xu, Hexin Jiang, and Xuefeng Liang. 2024. Leveraging knowledge of modality experts for incomplete multimodal learning. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 438–446.
- [49] Wenfang Yao, Kejing Yin, William K Cheung, Jia Liu, and Jing Qin. 2024. Drfuse: Learning disentangled representation for clinical multi-modal fusion with missing modality and modal inconsistency. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 16416–16424.
- [50] Lei Zhang, Zhiqiang Xie, Vaastav Anand, Ymir Vigfusson, and Jonathan Mace. 2023. The benefit of hindsight: Tracing {Edge-Cases} in distributed systems. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 321–339.
- [51] Shenglin Zhang, Pengxiang Jin, Zihan Lin, Yongqian Sun, Bicheng Zhang, Sibao Xia, Zhengdan Li, Zhenyu Zhong, Minghua Ma, Wa Jin, et al. 2023. Robust failure diagnosis of microservice system through multimodal data. *IEEE Transactions on Services Computing* 16, 6 (2023), 3851–3864.
- [52] Shenglin Zhang, Yingke Li, Jianjin Tang, Chenyu Zhao, Wenwei Gu, Yongqian Sun, and Dan Pei. 2025. Integrating GraphSAGE and Mamba for Self-Supervised Spatio-Temporal Fault Detection in Microservice Systems. In *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 323–334.
- [53] Zekun Zhang, Jian Wang, Bing Li, and Liuxiaoxiao Zhang. 2025. ReconRCA: Root Cause Analysis in Microservices with Incomplete Metrics. In *2025 IEEE International Conference on Web Services (ICWS)*. IEEE, 116–126.
- [54] Chenyu Zhao, Minghua Ma, Zhenyu Zhong, Shenglin Zhang, Zhiyuan Tan, Xiao Xiong, LuLu Yu, Jiayi Feng, Yongqian Sun, Yuzhi Zhang, et al. 2023. Robust multimodal failure detection for microservice systems. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5639–5649.